

Приватний заклад вищої освіти
Одеський технологічний університет «ШАГ»
Кафедра інформаційних технологій та фундаментальної підготовки

випускна кваліфікаційна робота бакалавра

**Інформаційна платформа розповсюдження мультимедійного контенту за
моделлю підписки (на прикладі сервісу Bazinga)**

на здобуття бакалаврського рівня вищої освіти
зі спеціальності «F3 Комп'ютерні науки»

Виконавці проекту:

№	П.І.Б.	Ролі	Група
1	Маймескул Єгор Олегович	Backend / Auth	КНП-221
2	Руденко Олена Костянтинівна	Backend / Content	КНП-221

Автор звіту: Маймескул Єгор Олегович

Керівник	Доцент кафедри ІТ та ФП, к.т.н., [Прізвище І. П. керівника]
----------	---

Одеса – 2025

АНОТАЦІЯ

УДК 004.9

М-15

Маймескул Є. О. Інформаційна платформа розповсюдження мультимедійного контенту за моделлю підписки (на прикладі сервісу Bazinga): підсистема ідентифікації, керування обліковими записами та платіжно-підписного обслуговування. Пояснювальна записка до дипломного проєкту на здобуття бакалаврського рівня вищої освіти зі спеціальності «ФЗ Комп'ютерні науки». — Одеса: ОТУШ, 2025. — 40 с.

Об'єкт розробки — серверна та клієнтська складові підсистеми, що виконують функції ідентифікації, автентифікації, керування багатопрофільним обліковим записом та платіжно-підписного обслуговування користувача у складі розподіленої інформаційної системи розповсюдження мультимедійного контенту за моделлю підписки.

Мета роботи — ефективне розроблення безпечної, відмовостійкої та масштабованої підсистеми керування обліковими записами, яка обробляє запити клієнтського застосунку щодо реєстрації, входу, багатопрофільного доступу та оформлення підписки в умовах неповної визначеності щодо доступності зовнішніх комунікаційних сервісів.

Методи дослідження та інструментарій — методи системного аналізу, порівняльного аналізу проєктних рішень та об'єктно-орієнтованого моделювання предметної галузі. Інструментальною базою обрано платформу ASP.NET Core 9 (мова C#) [1], технології доступу до даних Entity Framework Core 9 та LINQ [2] над системою керування базами даних MySQL 8 [3]; механізми автентифікації на основі JWT Bearer Tokens [4]; криптографічні засоби BCrypt та TOTP [6]; бібліотеки MailKit [15] і Stripe.NET [14]; клієнтський застосунок на React 18 [16] і TypeScript 5 [17] зі складанням засобами Vite 5 [18]; контейнеризація — Docker.

Результати та їх новизна — синтезовано структурні інформаційні моделі об'єктів предметної галузі (обліковий запис, профіль, одноразові токени) та впроваджено комунікаційний протокол на принципах REST. Новизною є не просте відтворення відомого ресурсу, а удосконалений варіант: поєднання безпарольної реєстрації, багатопрофільності з PIN-захистом, двофакторної автентифікації за стандартом TOTP та відмовостійкого платіжного потоку із градаційним зниженням функціональності (graceful degradation) при недоступності зовнішніх сервісів — таке поєднання відсутнє у розглянутих системах-аналогах.

Основні характеристики та показники — підсистема є складовою серверної частини з 13 контролерів і 8 інжектіваних сервісів; одноразові токени зберігаються виключно у вигляді SHA-256-згорток, паролі та PIN-коди — у вигляді адаптивних BCrypt-згорток; цільовий час відповіді ендпоінтів автентифікації — до 300 мс.

Інформація щодо впровадження — підсистему розгорнуто у середовищі Docker; обмін із клієнтом здійснюється комунікаційним протоколом REST поверх HTTP із підтвердженням маркером JWT; надсилання електронної пошти виконується через SMTP-сервіс або резервний журнальний режим.

Взаємозв'язок з іншими роботами — робота є складовою командного проєкту; підсистема взаємодіє з контентною складовою (каталог, агрегатор метаданих, потокове відео), що виконується другим учасником. Загальний висновок проєкту є сукупністю звітів усіх учасників.

Сфера застосування — вебсистеми розповсюдження цифрового контенту за моделлю підписки; підхід поширюється на суміжні класи систем е-комерції та е-послуг.

Актуальність роботи обумовлена не лише запитом замовника, а й загальною суспільною потребою у захищених засобах ідентифікації та електронного платежу, а також зростанням кількості автоматизованих атак на засоби автентифікації; результати придатні для повторного використання наступними розробниками як обґрунтований шаблон підсистеми облікових записів.

Висновки — спроектовано, реалізовано та випробувано підсистему облікових записів та платіжно-підписного обслуговування; усі поставлені у вступі задачі виконано. Перспективою є впровадження апаратних ключів автентифікації та зовнішніх провайдерів ідентифікації.

При складанні звіту використано 20 посилань на друковані та електронні джерела інформації.

ЗМІСТ

- АНОТАЦІЯ
 - ВСТУП
 - РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ
 - РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РЕАЛІЗАЦІЯ ПІДСИСТЕМИ ОБЛІКОВИХ ЗАПИСІВ
 - РОЗДІЛ 3. ВИПРОБУВАННЯ ТА ОЦІНКА ПІДСИСТЕМИ
 - ВИСНОВКИ
 - ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ
 - ДОДАТОК А. ТЕХНІЧНЕ ЗАВДАННЯ ПРОЄКТУ
-

ВСТУП

Сучасний стан предметної галузі визначається стрімким зростанням обсягів мультимедійного контенту та переходом споживання до інформаційних систем цифрової дистрибуції, що функціонують за моделлю «контент за підпискою». Невіддільною складовою таких систем є підсистема облікового запису: саме вона визначає, чи стане відвідувач передплатником, і саме вона несе відповідальність за захист персональних і платіжних даних. Підсистема ідентифікації об'єднує процеси реєстрації, входу, керування профілями та оформлення підписки у єдиний користувацький шлях, який має бути одночасно зручним, швидким і безпечним.

Світові підходи до розв'язання зазначених завдань передбачають відмову від класичної пари «логін + пароль» на користь безпарольних механізмів автентифікації (passwordless, magic link, одноразові коди), впровадження багаторівневих засобів захисту доступу (двофакторна автентифікація, PIN-захист окремих профілів), а також винесення обробки платіжних даних за межі власної інформаційної системи задля мінімізації області застосування галузевого стандарту захисту платіжних даних PCI DSS [11]. Аналіз провідних систем розповсюдження контенту (детально — у розділі 1) доводить, що жодна з них не поєднує одночасно єдиного облікового запису для різноманітного контенту, двофакторної автентифікації, PIN-захисту профілю та відмовостійкого платіжного потоку.

Актуальність роботи обумовлена загальною суспільною потребою у надійних та захищених засобах ідентифікації й електронного платежу, що є передумовою розвитку систем електронної комерції та електронних послуг. Реалізація захищеної підсистеми керування обліковими записами підвищує довіру користувача до сервісу та знижує ризики компрометації персональних даних, що набуває особливого значення з огляду на зростання кількості автоматизованих атак на засоби автентифікації.

Метою роботи є ефективне розроблення безпечної та відмовостійкої підсистеми керування обліковими записами, профілями й платіжно-підписною моделлю, здатної забезпечити коректну роботу всього користувацького шляху навіть за відмови окремих зовнішніх сервісів. Отримані результати можуть бути застосовані у вебсистемах розповсюдження контенту, а також у суміжних класах систем е-комерції та е-послуг.

Об'єктом дослідження виступає процес проєктування підсистеми ідентифікації, автентифікації та платіжно-підписного обслуговування користувача в межах розподіленої інформаційної системи. Предметом дослідження є архітектурні підходи, шаблони проєктування та криптографічні методи, які забезпечують безпеку, відмовостійкість і масштабованість цієї підсистеми. Завдання вирішуються в умовах неповної визначеності: зовнішні комунікаційні сервіси (платіжний шлюз, поштовий сервер) можуть бути не-

доступними, обмежувати кількість звернень або відповідати із затримкою, тому вихідні вимоги уточнювалися у процесі роботи, а технології порівнювалися й обиралися за визначеними критеріями, що надає роботі дослідницького характеру.

Для досягнення поставленої мети сформульовано такі завдання дослідження:

1. провести порівняльне дослідження наявних систем розповсюдження контенту за підпискою, виокремити їхні переваги й недоліки щодо засобів ідентифікації та захисту доступу, окреслити напрями вдосконалення;
2. синтезувати структурну інформаційну модель даних облікового запису, профілів та одноразових токенів; забезпечити ідемпотентне розгортання схеми бази даних;
3. реалізувати серверну частину підсистеми — комунікаційні інтерфейси (кінцеві точки REST API) та інфраструктурні сервіси автентифікації, поштового обслуговування й платіжної інтеграції;
4. впровадити багаторівневі засоби інформаційної безпеки: криптографічне зберігання облікових даних, безпарольну реєстрацію, відновлення доступу, PIN-захист профілів та двофакторну автентифікацію за стандартом TOTP;
5. реалізувати клієнтську частину підсистеми та узгодити стан клієнта і сервера засобами комунікаційного протоколу REST з підтвердженням маркером JWT;
6. провести випробування підсистеми за різних умов, зокрема за відмови зовнішніх сервісів, та оцінити відповідність технічному завданню.

Методи дослідження базуються на поєднанні методів системного аналізу, порівняльного аналізу аналогів та об'єктно-орієнтованого моделювання предметної галузі.

Взаємозв'язок з іншими складовими проєкту. Ця робота є складовою командного проєкту: цей звіт зосереджено на підсистемі облікових записів та платіжно-підписного обслуговування, яка взаємодіє з контентною підсистемою (каталог, агрегатор метаданих, потокове відео), що розробляється другим учасником команди. Початкові розділи, які описують загальну характеристику системи, є спільними для всієї команди; починаючи з розділу, присвяченого підсистемі облікових записів, матеріал викладено виключно як авторський.

РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ

1.1 Опис предметної галузі

Інформаційна система, що проектується (робоча назва — Bazinga), є розподіленою клієнт-серверною системою розповсюдження мультимедійного контенту за моделлю підписки. Система поєднує дві предметні складові у межах єдиного облікового запису: *Bazinga Comics* — каталог коміксів і манги з можливістю читання випусків онлайн, та *BazingaTV* — потоковий перегляд відеоконтенту (аніме, мультсеріали, супергеройські серіали). Спільним для обох складових є обліковий запис користувача з підтримкою декількох профілів, єдина модель підписки та узгоджений механізм оплати.

Підсистема, якій присвячено цей звіт, є спільною основою обох складових і охоплює увесь користувацький шлях — від першого введення електронної пошти на стартовій сторінці до обрання активного профілю після оформлення підписки. Серверний компонент відповідає вимогам сучасних систем дистрибуції контенту: безпека персональних і платіжних даних, мінімальне тертя під час реєстрації, стійкість до відмови зовнішніх сервісів та можливість горизонтального масштабування за рахунок stateless-архітектури.

Архітектурно систему побудовано за стилем «односторінковий застосунок (SPA) поверх комунікаційного протоколу REST». Серверна частина реалізована на платформі ASP.NET Core 9 [1] з об'єктно-реляційним відображенням Entity Framework Core [2] над MySQL [3]; клієнтська частина — мовою TypeScript [17] на основі бібліотеки React [16] зі складанням засобами Vite [18]. Підтвердження суб'єкта виконується маркером JWT [4], підписаним алгоритмом HMAC SHA-256.

1.2 Порівняльний аналіз наявних рішень

Для обґрунтування вибору взірцевої системи та визначення напрямів удосконалення виконано порівняльний аналіз найвідоміших систем розповсюдження контенту за ознаками, що стосуються ідентифікації користувача та захисту доступу (табл. 1.1). Як значення ознаки використано «+» (наявна), «—» (відсутня) або «частково» (наявна у спрощеному вигляді).

Таблиця 1.1 — Порівняння систем розповсюдження контенту за засобами ідентифікації та захисту доступу

Ознака / Система	Netflix	Crunchyroll	ComiXology	Bazinga
Єдиний обліковий запис для відео та коміксів/манги	—	—	—	+
Безпарольна реєстрація за магічним посиланням	+	—	—	+

Ознака / Система	Netflix	Crunchyroll	ComiXology	Bazinga
Багатопрофільність (до 5 профілів)	+	частково	—	+
PIN-захист окремого профілю	+	—	—	+
Двофакторна автентифікація (TOTP)	—	—	частково	+
Підтвердження номера телефону	+	частково	—	+
Відмовостійкий платіжний потік із резервним режимом	—	—	—	+

Як свідчить таблиця 1.1, жодна із систем не поєднує усіх зазначених ознак одночасно, що підтверджує диференційність обраних критеріїв. Найбільше переваг серед аналогів має Netflix, який і обрано за взірцеву систему щодо моделі багатопрофільного доступу. Водночас жодна з наявних систем не поєднує єдиного облікового запису для різноманітного контенту з двофакторною автентифікацією, PIN-захистом профілю та відмовостійким платіжним потоком — саме усунення цих «мінусів» становить предмет удосконалення у даній роботі.

1.3 Постановка задачі розроблення

Проведений аналіз засвідчив, що готові продукти не відповідають встановленим вимогам: вони орієнтовані на один тип контенту, не поєднують повного набору засобів захисту доступу або прив'язані до єдиного постачальника. З огляду на це, розроблення власної підсистеми є доцільним, оскільки дозволяє усунути виявлені вади шляхом поєднання безпарольної реєстрації, багатопрофільності з PIN-захистом, двофакторної автентифікації та відмовостійкого платіжного обслуговування у межах єдиного облікового запису.

До функціональних вимог підсистеми належать: безпарольна реєстрація за одноразовим посиланням із миттєвою перевіркою зайнятості адреси; вхід за паролем або за одноразовим посиланням; відновлення паролю; двофакторна автентифікація за стандартом TOTP; багатопрофільність (до п'яти профілів) з PIN-захистом окремого профілю; оформлення підписки з прив'язкою платіжного інструменту; керування персональними даними та номером телефону. До нефункціональних вимог належать: продуктивність (час відповіді ендпоінтів автентифікації — до 300 мс); безпека (зберігання паролів і токенів виключно у вигляді криптографічних згорток, підпис маркера сесії, обмеження темпу й тривалості сесій згідно з рекомендаціями [9, 10]); масштабованість (stateless-архітектура серверної частини); адаптивність (коректна робота у сучасних браузерях та на пристроях різних класів). Повний текст технічного завдання наведено у Додатку А.

Висновок до розділу. У межах роботи реалізується підсистема облікових записів удосконаленої системи розповсюдження контенту, яка усуває виявлені недоліки наявних аналогів. Використання готового стороннього рішення є недоцільним через орієнтацію

аналогів на один тип контенту та брак повного набору засобів захисту доступу. Розроблена підсистема є відповіддю на суспільний попит щодо розвитку захищених вітчизняних електронних сервісів. Це обґрунтовує предмет і задачі подальшого проектування, викладеного у розділі 2.

РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РЕАЛІЗАЦІЯ ПІДСИСТЕМИ ОБЛІКОВИХ ЗАПИСІВ

2.1 Обґрунтування вибору технологій

Вибір технологій виконувався дослідницьким методом: для кожного компонента опрацьовувалися альтернативи й обирався варіант, що максимально відповідає заданим критеріям — швидкодії, захищеності, здатності до масштабування та простоті супроводу. Серед платформ серверного розроблення аналізувалися Node.js, Spring (Java) та ASP.NET Core. Перевагу надано ASP.NET Core 9 (мова C#) [1] як строго типізований, високопродуктивній платформі зі зрілою екосистемою, вбудованою інверсією залежностей та розвиненими засобами тестування.

Для об'єктно-реляційного відображення обрано Entity Framework Core 9 [2] з провайдером Pomelo для MySQL, що забезпечує типобезпечні запити мовою LINQ та керування схемою бази даних. Для криптографічного перетворення паролів розглянуто алгоритми PBKDF2 (RFC 2898) [10], BCrypt та Argon2; обрано BCrypt як адаптивний алгоритм із налаштовуваним фактором роботи, рекомендований OWASP [9] і реалізований у зрілій бібліотеці. Для підтвердження сесії обрано маркер JWT (RFC 7519) [4], що не потребує серверного стану й узгоджується зі stateless-архітектурою; для несення маркера у запитах застосовано схему Bearer (RFC 6750) [5]. Для двофакторної автентифікації обрано алгоритм TOTP (RFC 6238) [6], сумісний із поширеними застосунками-автентифікаторами; одноразові секрети кодуються за схемою Base32 (RFC 4648) [7]. Поштове обслуговування реалізовано бібліотекою MailKit [15], платіжну інтеграцію — офіційною бібліотекою Stripe.NET [14].

Для клієнтської частини обрано React 18 [16] з мовою TypeScript [17] зі складанням засобами Vite [18]. Статична типізація знижує кількість помилок на етапі компіляції, а компонентна модель забезпечує повторне використання елементів інтерфейсу. Маршрутизацію реалізовано бібліотекою React Router [19], керування серверним станом — бібліотекою TanStack Query [20].

2.2 Архітектура підсистеми та принципи SOLID

Підсистему спроектовано за тришаровою організацією з чітким поділом відповідальностей: рівень подання (контролери), рівень бізнес-логіки (інжектовані сервіси) та рівень доступу до даних (контекст бази даних та сутності). Така організація забезпечує тестованість логіки незалежно від бази даних та можливість підміни реалізацій без зміни контролерів.

Проектні рішення підпорядковано принципам SOLID. Принцип єдиної відповідальності (S) реалізовано закріпленням за кожним контролером однієї ресурсної області (автентифікація, профілі, підписи). Принцип відкритості/закритості (O) забезпечено тим, що додавання нового засобу обслуговування (наприклад, нової реалізації відправлення пошти) не змінює наявних контролерів, а підключається через інверсію залежностей. Принцип підстановки Лісков (L) втілено інтерфейсом `IMailSender` із двома взаємозамінними реалізаціями. Принцип розділення інтерфейсів (I) реалізовано вузькими інтерфейсами `IPasswordHasher`, `IJwtService`, `IBillingService`. Принцип інверсії залежностей (D) забезпечено тим, що контролери залежать від абстракцій, а конкретні реалізації підключаються через контейнер інверсії залежностей.

Серед шаблонів проектування застосовано: `Strategy` — для взаємозамінних реалізацій поштового та платіжного сервісів; `Decorator` — для обгортання HTTP-клієнта платіжного шлюзу механізмами тайм-ауту та повтору; `Repository` та `Unit of Work` — неявно через `DbSet<T>` та `SaveChangesAsync` контексту `Entity Framework Core`; `DTO` — для відокремлення моделей передачі даних від доменних сутностей.

2.3 Структурна модель даних

Синтезовано структурну інформаційну модель, центральним вузлом якої є сутність *User* (обліковий запис). До неї каскадно прив'язані сутність *Profile* (профіль) та одноразові токени різних призначень. Модель нормалізовано до третьої нормальної форми.

Сутність *User* містить унікальні атрибути імені користувача та електронної пошти, `BCrypt`-згортку паролю, номер телефону, тип і термін підписки, а також атрибути двофакторної автентифікації — прапорець активації та секрет TOTP, позначений як прихований при серіалізації. Сутність *Profile* містить посилання на власника, ім'я, аватар, прапорці кореневого та дитячого профілю та (опціонально) `BCrypt`-згортку PIN-коду. Одноразові токени (реєстрації, входу, відновлення паролю, виклику двофакторної автентифікації) зберігаються виключно у вигляді SHA-256-згорток з атрибутами терміну дії та факту використання; оригінальне значення посилання у сховищі відсутнє.

Ідемпотентне розгортання схеми реалізовано викликами `CREATE TABLE IF NOT EXISTS` та умовним додаванням стовпців, що дозволяє оновлювати наявні екземпляри без ручного втручання та без втрати даних. Приклад 1 ілюструє генерацію одноразового токена та його незворотне згортання, що є основою всіх безпарольних сценаріїв.

Приклад 1 — Генерація одноразового токена та його незворотне SHA-256-згортання (`Security/SignupTokens.cs`)

```
public static class SignupTokens
{
    public static string Generate()
    {
        var bytes = RandomNumberGenerator.GetBytes(32); // 256 біт ентропії
        return Convert.ToBase64String(bytes)
    }
}
```

```

        .Replace('+', '-').Replace('/', '_').TrimEnd('=');
    }

    public static string Hash(string token)
    {
        var bytes = SHA256.HashData(Encoding.UTF8.GetBytes(token));
        return Convert.ToHexString(bytes); // у БД зберігається лише згортка
    }
}

```

2.4 Безпарольна реєстрація за магічним посиланням

Класична реєстрація через логін і пароль створює зайві бар'єри для користувача, не гарантує миттєвої верифікації електронної пошти та ускладнює роботу з платіжними даними. Тому реалізовано безпарольний підхід із використанням одноразового посилання (magic link): користувач вказує лише електронну пошту, після чого проходить увесь процес реєстрації та автоматично входить у систему. Електронна пошта при цьому верифікується автоматично самим фактом переходу за посиланням.

Сценарій реалізують чотири кінцеві точки контролера автентифікації: миттєва перевірка зайнятості адреси; ініціювання реєстрації з генерацією токена та надси­ланням листа; верифікація токена; фіналізація реєстрації зі створенням облікового запису й видачею маркера сесії. Перед надси­ланням нового листа всі попередні живі токени для цієї адреси інвалідуються, тому дійсним лишається лише найсвіжіше посилання (Приклад 2). Це водночас протидіє накопиченню активних посилань та спрощує діагностику.

Приклад 2 — Ініціювання безпарольної реєстрації: інвалідація попередніх токенів і надси­лання посилання (Controllers/AuthController.cs)

```

[HttpPost("signup/start")]
public async Task<IActionResult> SignupStart([FromBody] SignupStartRequest req,
Cancellation token ct)
{
    var email = req.Email?.Trim();
    if (string.IsNullOrEmpty(email)) return BadRequest("Email is required.");
    if (await _db.Users.AnyAsync(u => u.Email == email, ct))
        return Conflict("An account with this email already exists.");

    // Інвалідуємо всі попередні живі токени – діє лише найсвіжіше посилання.
    await _db.SignupTokens
        .Where(t => t.Email == email && t.ConsumedAt == null)
        .ExecuteUpdateAsync(s => s.SetProperty(t => t.ConsumedAt, DateTime.UtcNow), ct);

    var rawToken = SignupTokens.Generate();
    _db.SignupTokens.Add(new SignupToken
    {
        Email = email!,
        TokenHash = SignupTokens.Hash(rawToken),
        ExpiresAt = DateTime.UtcNow.AddMinutes(SignupTokenLifetimeMinutes),
        OptOutMarketing = req.OptOutMarketing
    });
    await _db.SaveChangesAsync(ct);

    var baseUrl = _emailOptions.PublicBaseUrl.TrimEnd('/');
    var link = $"{baseUrl}/signup/complete?token={Uri.EscapeDataString(rawToken)}";
    await _emailSender.SendAsync(email!, "You're almost there!", BuildSignupEmail(link),
ct);
}

```

```
return NoContent();
}
```

Надсилання листа делеговано інтерфейсу `ISender`, що має дві взаємозамінні реалізації (Strategy): виробничу — через SMTP засобами MailKit — та резервну, яка друкує згенероване посилання у журнал. Резервна реалізація забезпечує відтворюваність сценарію реєстрації у середовищах без поштового сервера, що особливо корисно під час демонстрації (Приклад 3).

Приклад 3 — Дві взаємозамінні реалізації відправлення пошти (Strategy) та їх вибір у контейнері залежностей (Program.cs)

```
// Виробничий варіант – реальний SMTP; резервний – друк посилання у журнал.
if (emailHostConfigured)
    builder.Services.AddSingleton<ISender, SmtEmailSender>();
else
    builder.Services.AddSingleton<ISender, LoggingEmailSender>();
```

Фіналізація реєстрації перевіряє стан токена (живий / використаний / прострочений), створює обліковий запис із обраним тарифом, атомарно формує кореневий профіль і повертає підписаний маркер сесії (Приклад 4). Атомарне створення кореневого профілю безпосередньо при реєстрації усуває потребу у його ліниво-відкладеному створенні та запобігає виникненню стану гонитви (див. п. 2.7).

Приклад 4 — Фіналізація реєстрації: перевірка токена, створення облікового запису й кореневого профілю (Controllers/AuthController.cs)

```
[HttpPost("signup/complete")]
public async Task<ActionResult<AuthResponse>> SignupComplete([FromBody]
    SignupCompleteRequest req, CancellationToken ct)
{
    if (string.IsNullOrEmpty(req.Token)) return BadRequest("Token is required.");
    if (string.IsNullOrEmpty(req.Password) || req.Password!.Length < 6)
        return BadRequest("Password must be at least 6 characters.");

    var entity = await _db.SignupTokens.FirstOrDefaultAsync(t => t.TokenHash ==
        SignupTokens.Hash(req.Token!), ct);
    if (entity is null) return BadRequest("Invalid sign-up link.");
    if (entity.ConsumedAt is not null) return BadRequest("This link has already been
        used.");
    if (entity.ExpiresAt < DateTime.UtcNow) return BadRequest("This link has expired.");

    var (subscriptionType, subscriptionExpiration) = (req.Plan ??
        "trial").Trim().ToLowerInvariant() switch
    {
        "unlimited" => ("Unlimited",
            (DateOnly?)DateOnly.FromDateTime(DateTime.UtcNow.AddDays(30))),
        "comics" => ("Comics",
            (DateOnly?)DateOnly.FromDateTime(DateTime.UtcNow.AddDays(30))),
        "tv" => ("TV",
            (DateOnly?)DateOnly.FromDateTime(DateTime.UtcNow.AddDays(30))),
        _ => ("Trial",
            (DateOnly?)DateOnly.FromDateTime(DateTime.UtcNow.AddDays(7))),
    };

    var user = new User
    {
        Email = entity.Email,
```

```

        Username = await GenerateUniqueUsername(entity.Email, ct),
        Password = _hasher.Hash(req.Password!),
        Role = "USER",
        SubscriptionType = subscriptionType,
        SubscriptionExpiration = subscriptionExpiration
    };
    _db.Users.Add(user);
    entity.ConsumedAt = DateTime.UtcNow;
    await _db.SaveChangesAsync(ct);

    EnsureRootProfile(user); // атомарне створення кореневого профілю
    await _db.SaveChangesAsync(ct);
    return Ok(BuildResponse(user));
}

```

Поряд із реєстрацією за тією ж моделлю реалізовано безпарольний вхід та відновлення паролю: окремі кінцеві точки видають одноразове посилання, перехід за яким відповідно входить у систему або дозволяє задати новий пароль. Усі три сценарії спираються на спільний механізм одноразових SHA-256-згорнутих токенів (Приклад 1), що уніфікує модель безпеки.

2.5 Криптографічний захист облікових даних

Засоби інформаційної безпеки утворюють багаторівневу модель. Паролі та PIN-коди перетворюються адаптивним алгоритмом BCrypt: відновлення оригіналу зі згортки є обчислювально складним, а налаштовуваний фактор роботи дозволяє підвищувати вартість обчислення згортки в міру зростання потужності апаратних засобів (Приклад 5). Винесення алгоритму за інтерфейс `IPasswordHasher` дозволяє змінити його у майбутньому без зміни коду контролерів.

Приклад 5 — Адаптивне BCrypt-згортання та безпечна перевірка паролів і PIN-кодів (`Services/BCryptPasswordHasher.cs`)

```

public class BCryptPasswordHasher : IPasswordHasher
{
    public string Hash(string password) => BCrypt.Net.BCrypt.HashPassword(password);

    public bool Verify(string password, string hash)
    {
        try { return BCrypt.Net.BCrypt.Verify(password, hash); }
        catch { return false; } // некоректна згортка не призводить до
    }
}

```

Сесія підтверджується маркером JWT, підписаним алгоритмом HMAC SHA-256. До маркера додаються клейми ідентифікатора, імені, електронної пошти та ролі; термін дії обмежено (Приклад 6). Маркер передається у заголовок за схемою Bearer, що за дизайном унеможливорює атаки підміни міжсайтового запиту (CSRF), оскільки маркер не зберігається у файлах cookie.

Приклад 6 — Формування й підписання маркера сесії JWT алгоритмом HMAC SHA-256 (`Services/JwtService.cs`)

```

public string GenerateToken(User user)
{
    var role = string.IsNullOrEmpty(user.Role) ? "USER" :
user.Role!.ToUpperInvariant();
    var claims = new List<Claim>
    {
        new(JwtRegisteredClaimNames.Sub, user.Email),
        new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
        new(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new(ClaimTypes.Name, user.Username),
        new(ClaimTypes.Email, user.Email),
        new(ClaimTypes.Role, role),
    };
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_options.Secret));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
    var token = new JwtSecurityToken(
        issuer: _options.Issuer, audience: _options.Audience, claims: claims,
        notBefore: DateTime.UtcNow,
        expires: DateTime.UtcNow.AddMilliseconds(_options.ExpirationMs),
        signingCredentials: creds);
    return new JwtSecurityTokenHandler().WriteToken(token);
}

```

Під час випробування (розділ 3) підтверджено, що оригінальні токени не зберігаються у базі даних, мають обмежений термін дії та не можуть бути використані повторно, а паролі зберігаються виключно у вигляді згорток. Унаслідок цього навіть компрометація бази даних не надає доступу до облікових записів користувачів або їхніх активних сесій.

2.6 Двофакторна автентифікація за стандартом TOTP

Для підвищення захищеності облікового запису впроваджено двофакторну автентифікацію за стандартом TOTP (RFC 6238) [6], сумісну з поширеними застосунками-автентифікаторами. Алгоритм реалізовано власними засобами без додаткових залежностей: реалізовано генерацію секрету у кодуванні Base32 (RFC 4648) [7], формування `otpauth-`URI для QR-коду та перевірку коду з допуском відхилення годинника у межах одного інтервалу. Перевірку коду наведено у Прикладі 7.

Приклад 7 — Перевірка коду TOTP із допуском відхилення годинника та обчислення значення (`Security/Totp.cs`)

```

public static bool Verify(string? secret, string? code, int window = 1)
{
    if (string.IsNullOrEmpty(secret) || string.IsNullOrEmpty(code)) return false;
    code = code.Trim();
    if (code.Length != Digits || !code.All(char.IsDigit)) return false;

    byte[] key;
    try { key = Base32Decode(secret); } catch { return false; }

    var counter = DateTimeOffset.UtcNow.ToUnixTimeSeconds() / PeriodSeconds;
    for (var offset = -window; offset <= window; offset++) // ±1 інтервал (±30 с)
        if (FixedTimeEquals(Compute(key, counter + offset), code)) return true;
    return false;
}

private static string Compute(byte[] key, long counter)
{

```

```

var counterBytes = BitConverter.GetBytes(counter);
if (BitConverter.IsLittleEndian) Array.Reverse(counterBytes);
using var hmac = new HMACSHA1(key);
var hash = hmac.ComputeHash(counterBytes);
var off = hash[^1] & 0x0F; // динамічне зрізання
var binary = ((hash[off] & 0x7F) << 24) | ((hash[off + 1] & 0xFF) << 16)
             | ((hash[off + 2] & 0xFF) << 8) | (hash[off + 3] & 0xFF);
return (binary % (int)Math.Pow(10, Digits)).ToString().PadLeft(Digits, '0');
}

```

За увімкненої двофакторної автентифікації перший фактор (пароль або одноразове посилання) не завершує сесію, а повертає одноразовий виклик (challenge), який обмінюється на маркер лише після перевірки коду (Приклад 8). Виклик зберігається у вигляді SHA-256-згортки з обмеженим терміном дії та ознакою одноразовості; за неправильного коду виклик залишається живим для повторної спроби в межах терміну дії.

Приклад 8 — Обмін одноразового виклику двофакторної автентифікації на маркер сесії (Controllers/AuthController.cs)

```

[HttpPost("2fa/login-verify")]
public async Task<ActionResult<AuthResponse>> TwoFactorLoginVerify(
    [FromBody] TwoFactorLoginVerifyRequest req, CancellationToken ct)
{
    if (string.IsNullOrEmpty(req.ChallengeToken)) return BadRequest("Challenge token is required.");
    if (string.IsNullOrEmpty(req.Code)) return BadRequest("Code is required.");

    var hash = SignupTokens.Hash(req.ChallengeToken!);
    var challenge = await _db.TwoFactorChallenges.FirstOrDefaultAsync(c => c.TokenHash == hash, ct);
    if (challenge is null || challenge.ConsumedAt is not null || challenge.ExpiresAt < DateTime.UtcNow)
        return BadRequest("This verification step has expired. Please sign in again.");

    var user = await _db.Users.FirstOrDefaultAsync(u => u.Id == challenge.UserId, ct);
    if (user is null || string.IsNullOrEmpty(user.TwoFactorSecret)) return
        BadRequest("Account not found.");
    if (!Totp.Verify(user.TwoFactorSecret, req.Code)) return BadRequest("Incorrect code. Try again.");

    challenge.ConsumedAt = DateTime.UtcNow; // одноразовість виклику
    await _db.SaveChangesAsync(ct);
    return Ok(BuildResponse(user)); // лише тут видається маркер
}

```

2.7 Багатопрофільність та PIN-захист профілю

Підсистема реалізує багатопрофільну модель за зразком сімейного плану Netflix: один обліковий запис містить до п'яти профілів. Контролер профілів забезпечує повний цикл операцій над профілями з контролем бізнес-обмежень — ліміту кількості профілів та заборони видалення кореневого профілю. Окремий профіль може бути захищений чотиризначним PIN-кодом, згортка якого перевіряється перед обранням профілю; кореневий профіль може скидати PIN-коди дочірніх профілів, що реалізує функцію батьківського контролю (Приклад 9).

Приклад 9 — Встановлення та перевірка PIN-коду профілю з батьківським контролем (Controllers/ProfilesController.cs)

```
[HttpPut("{id:long}/pin")]
public async Task<IActionResult> SetPin(long id, [FromBody] SetPinRequest req,
CancellationToken ct)
{
    var user = await CurrentUser.GetAsync(User, _db, ct);
    if (user is null) return Unauthorized();
    var profile = await _db.Profiles.FirstOrDefaultAsync(p => p.Id == id && p.UserId ==
user.Id, ct);
    if (profile is null) return NotFound();
    if (string.IsNullOrEmpty(req.Pin) || !PinFormat.IsMatch(req.Pin!)) // рівно 4 цифри
        return BadRequest("PIN must be exactly 4 digits.");

    // Заміна наявного PIN потребує поточного – окрім кореневого профілю (батьківський
    контроль).
    if (!string.IsNullOrEmpty(profile.PinHash) && !await CanBypassPinAsync(user, ct))
        if (string.IsNullOrEmpty(req.CurrentPin) || !_hasher.Verify(req.CurrentPin!,
profile.PinHash))
            return BadRequest("Current PIN does not match.");

    profile.PinHash = _hasher.Hash(req.Pin!);
    await _db.SaveChangesAsync(ct);
    return NoContent();
}
```

Окремим викликом стала Netflix-подібна поведінка, за якої користувач після нового входу залишається авторизованим, але обирає профіль наново. Для цього на клієнті розділено збереження стану: відомості облікового запису та маркер сесії зберігаються довгостроково, а ідентифікатор активного профілю — лише на час сесії (Приклад 12). Завдяки цьому при кожному новому відвідуванні користувач знову бачить екран обрання профілю «Who's watching?».

Особливістю середовища React є подвійний виклик ефектів у режимі розробки (StrictMode), що за лінивого створення кореневого профілю призводив до появи двох профілів «Main» (стан гонитви). Проблема усунено трирівневим захистом: кореневий профіль створюється атомарно при реєстрації (Приклад 4); для наявних облікових записів передбачено резервне створення з перехопленням винятку оновлення бази даних і подальшою дедуплікацією; на клієнті усунено зайвий повторний запит. Випробування підтвердило, що для кожного облікового запису створюється рівно один кореневий профіль як для нових, так і для наявних користувачів.

2.8 Відмовостійка інтеграція платіжного шлюзу

Інтеграцію з платіжним шлюзом побудовано за принципом мінімізації області застосування стандарту PCI DSS [11]: чутливі дані картки (номер, термін дії, код перевірки) не потрапляють на сервер системи, а збираються офіційним компонентом шлюзу. Сервер створює об'єкти `Customer` і `SetupIntent` та повертає клієнту одноразовий секрет, після чого клієнт завершує прив'язку картки безпосередньо у шлюзі (Приклад 10).

Приклад 10 — Створення об'єктів платіжного шлюзу з обмеженим тайм-аутом і повтором (Decorator) (Services/StripeBillingService.cs)

```
public StripeBillingService(IOptions<StripeOptions> options, ILogger<StripeBillingService>
logger)
{
    _options = options.Value;
    if (IsConfigured)
    {
        // Тісний тайм-аут і один повтор: заблоковане з'єднання падає швидко, а не
        «зависає».
        var httpClient = new HttpClient { Timeout = TimeSpan.FromSeconds(12) };
        _stripeClient = new StripeClient(_options.SecretKey,
            httpClient: new SystemNetHttpClient(httpClient, maxNetworkRetries: 1));
    }
}

public async Task<SetupIntentResult> CreateSetupIntentAsync(string email, CancellationToken
ct = default)
{
    var customer = await new CustomerService(_stripeClient)
        .CreateAsync(new CustomerCreateOptions { Email = email }, cancellationTokens: ct);
    var intent = await new SetupIntentService(_stripeClient).CreateAsync(
        new SetupIntentCreateOptions
        {
            Customer = customer.Id,
            PaymentMethodTypes = new List<string> { "card" },
            Usage = "off_session",
        }, cancellationTokens: ct);
    return new SetupIntentResult(intent.ClientSecret, customer.Id); // секрет картки на
сервер не потрапляє
}
```

Звернення до шлюзу обгорнуто механізмами тайм-ауту (12 секунд) та одного автоматичного повтору; помилки шлюзу відображаються у відповідні коди стану HTTP. За недоступності шлюзу клієнт переходить у резервний режим із мок-формою, що дозволяє завершити сценарій реєстрації — це реалізація градаційного зниження функціональності в умовах невизначеності щодо доступності зовнішнього сервісу. Унаслідок цього платіжні дані обробляються безпосередньо шлюзом, а процес реєстрації коректно завершується навіть за збоїв зовнішнього сервісу.

2.9 Клієнтська частина та керування станом

Клієнтську частину реалізовано як сукупність сторінок безпарольної реєстрації, входу, відновлення паролю, селектора та редактора профілю й особистого кабінету. Завершення реєстрації реалізовано як скінченний автомат із чотирма станами (Welcome → Password → Plan → Card). Стан автентифікації централізовано у контексті `AuthContext` із диференційованим зберіганням за терміном дії (Приклад 11).

Приклад 11 — Диференційоване зберігання стану та обробка виклику двофакторної автентифікації на клієнті (contexts/AuthContext.tsx)

```
// Відомості акаунта та маркер – довгостроково; активний профіль – лише на сесію.
const [token, setToken] = useState<string | null>(() => localStorage.getItem("auth_token"));
const [currentProfileId, setCurrentProfileId] = useState<number | null>(() => {
    const raw = sessionStorage.getItem(CURRENT_PROFILE_KEY);
```

```

    return raw ? Number(raw) : null;
  });

  // Перший фактор: якщо увімкнено 2FA — повертаємо виклик замість завершення сесії.
  const applyFirstFactor = (response: AuthApiResponse): LoginOutcome => {
    if (response.twoFactorRequired && response.challengeToken) {
      return { twoFactorRequired: true, challengeToken: response.challengeToken };
    }
    handleAuth(response.token, response);
    return { twoFactorRequired: false };
  };

```

Захист маршрутів реалізовано охоронними компонентами, які запобігають доступу до контенту без підтвердженої сесії та обраного профілю, а також перенаправляють користувачів із простроченим пробним тарифом на сторінку обрання плану (Приклад 12).

Приклад 12 — Охоронний компонент маршрутизації клієнта (App.tsx)

```

const RequireProfile = ({ children }: { children: React.ReactNode }) => {
  const { user, currentProfile, trialExpired, profiles, profilesLoading } = useAuth();
  if (!user) return <Navigate to="/auth" replace />;
  if (!currentProfile) {
    if (usingStoredProfile(profiles, profilesLoading)) return <ProfilesLoading />;
    return <Navigate to="/profiles" replace />; // не обрано профіль
  }
  if (trialExpired) return <Navigate to="/bazinga-unlimited?from=trial-expired" replace />;
  return <>{children}</>;
};

```

2.10 Безпарольний вхід, відновлення доступу та допоміжні сценарії

Окрім реєстрації, за спільною моделлю одноразових токенів реалізовано безпарольний вхід та відновлення паролю. Кінцева точка безпарольного входу перевіряє стан токена та, у разі його дійсності, визначає, чи увімкнено для облікового запису двофакторну автентифікацію: якщо так — повертається одноразовий виклик (див. п. 2.6), інакше одразу видається маркер сесії (Приклад 13). Така побудова уніфікує парольний і безпарольний шляхи входу щодо другого фактора.

Приклад 13 — Безпарольний вхід: перевірка токена та розгалуження за двофакторною автентифікацією (Controllers/AuthController.cs)

```

[HttpPost("signin/verify")]
public async Task<ActionResult<AuthResponse>> SigninVerify([FromBody] SigninVerifyRequest req, CancellationToken ct)
{
    if (string.IsNullOrEmpty(req.Token)) return BadRequest("Token is required.");
    var hash = SignupTokens.Hash(req.Token!);
    var entity = await _db.SigninTokens.FirstOrDefaultAsync(t => t.TokenHash == hash, ct);
    if (entity is null) return BadRequest("Invalid sign-in link.");
    if (entity.ConsumedAt is not null) return BadRequest("This link has already been used.");
    if (entity.ExpiresAt < DateTime.UtcNow) return BadRequest("This link has expired.");

    var user = await _db.Users.FirstOrDefaultAsync(u => u.Email == entity.Email, ct);
    if (user is null) return BadRequest("Account not found.");
    entity.ConsumedAt = DateTime.UtcNow; // токен одноразовий
    await _db.SaveChangesAsync(ct);
}

```

```

    if (RequiresTwoFactor(user)) return Ok(await ChallengeResponseAsync(user, ct));
    return Ok(BuildResponse(user));
}

```

Відновлення паролю реалізовано окремою кінцевою точкою, що приймає одноразовий токен відновлення та новий пароль; за дійсності токена пароль перезаписується у вигляді нової BCrypt-згортки, а токен позначається використаним (Приклад 14). Запит на надсилання листа відновлення навмисно не повідомляє, чи існує обліковий запис із зазначеною адресою, щоб не розкривати наявність акаунтів — це відповідає рекомендаціям щодо протидії перерахуванню облікових записів [9].

Приклад 14 — Відновлення паролю за одноразовим посиланням (Controllers/AuthController.cs)

```

[HttpPost("reset-password")]
public async Task<IActionResult> ResetPassword([FromBody] ResetPasswordRequest req,
CancellationTokens ct)
{
    if (string.IsNullOrEmpty(req.Token)) return BadRequest("Token is required.");
    if (string.IsNullOrEmpty(req.NewPassword) || req.NewPassword!.Length < 6)
        return BadRequest("Password must be at least 6 characters.");

    var entity = await _db.PasswordResetTokens
        .FirstOrDefaultAsync(t => t.TokenHash == SignupTokens.Hash(req.Token!), ct);
    if (entity is null) return BadRequest("Invalid reset link.");
    if (entity.ConsumedAt is not null) return BadRequest("This link has already been
used.");
    if (entity.ExpiresAt < DateTime.UtcNow) return BadRequest("This link has expired.");

    var user = await _db.Users.FirstOrDefaultAsync(u => u.Email == entity.Email, ct);
    if (user is null) return BadRequest("Account not found.");
    user.Password = _hasher.Hash(req.NewPassword!); // новий пароль – у вигляді згортки
    entity.ConsumedAt = DateTime.UtcNow;
    await _db.SaveChangesAsync(ct);
    return NoContent();
}

```

Окремої уваги потребувало запобігання стану гонитви при створенні кореневого профілю. Середовище React у режимі розробки виконує ефекти двічі (StrictMode), унаслідок чого ліниве створення кореневого профілю призводило до появи двох профілів «Main». Проблема усунуто трирівнево: основним механізмом є атомарне створення кореневого профілю безпосередньо при реєстрації (Приклад 4); для наявних облікових записів передбачено резервне створення з перехопленням винятку оновлення бази даних та подальшою дедуплікацією корневих профілів зі збереженням найранішого (Приклад 15); на клієнті усунуто зайвий повторний запит списку профілів.

Приклад 15 — Резервне створення кореневого профілю та дедуплікація (Controllers/ProfilesController.cs)

```

if (profiles.Count == 0)
{
    _db.Profiles.Add(new Entities.Profile
    {
        UserId = user.Id,
        Name = string.IsNullOrEmpty(user.FirstName) ? user.Username : user.FirstName!,
    });
}

```

```

        AvatarColor = DefaultAvatarColor, IsRoot = true, IsKids = false,
    });
    try { await _db.SaveChangesAsync(ct); }
    catch (DbUpdateException) { /* паралельний запит уже створив – підхопимо нижче */ }

    profiles = await _db.Profiles.Where(p => p.UserId == user.Id)
        .OrderByDescending(p => p.IsRoot).ThenBy(p => p.Id).ToListAsync(ct);

    // Якщо два паралельні запити встигли створити по кореневому – лишаємо найраніший.
    var roots = profiles.Where(p => p.IsRoot).OrderBy(p => p.Id).ToList();
    if (roots.Count > 1)
    {
        _db.Profiles.RemoveRange(roots.Skip(1));
        await _db.SaveChangesAsync(ct);
    }
}

```

Платіжний крок майстра реєстрації обслуговує окрема кінцева точка, яка створює об'єкти платіжного шлюзу та повертає клієнту одноразовий секрет; за відсутності налаштованого шлюзу вона повертає ознаку резервного режиму, а помилки шлюзу відображає у коді стану 502/504 (Приклад 16). Це доповнює відмовостійкий механізм, описаний у п. 2.8.

Приклад 16 — Кінцева точка платіжного наміру з резервним режимом і відображенням помилок (Controllers/AuthController.cs)

```

[HttpPost("signup/billing-intent")]
public async Task<ActionResult<BillingIntentResponse>> SignupBillingIntent(
    [FromBody] BillingIntentRequest req, CancellationToken ct)
{
    var entity = await _db.SignupTokens
        .FirstOrDefaultAsync(t => t.TokenHash == SignupTokens.Hash(req.Token!), ct);
    if (entity is null || entity.ConsumedAt is not null || entity.ExpiresAt <
        DateTime.UtcNow)
        return BadRequest("Invalid or expired sign-up link.");

    if (!_billing.IsConfigured)
        return Ok(new BillingIntentResponse { StripeConfigured = false }); // резервний
    режим
    try
    {
        var intent = await _billing.CreateSetupIntentAsync(entity.Email, ct);
        return Ok(new BillingIntentResponse
        {
            StripeConfigured = true, ClientSecret = intent.ClientSecret,
            CustomerId = intent.CustomerId, PublishableKey = _billing.PublishableKey,
        });
    }
    catch (Stripe.StripeException ex) { return StatusCode(502, ex.Message); }
    catch (OperationCanceledException) { return StatusCode(504, "Stripe timeout."); }
}

```

На клієнті звернення до кінцевих точок безпеки інкапсульовано у тонкому шарі функцій, що приховує деталі комунікаційного протоколу від компонентів інтерфейсу (Приклад 17). Такий підхід спрощує супровід і тестування клієнтської частини.

Приклад 17 — Клієнтський шар звернень до засобів двофакторної автентифікації (lib/auth.ts)

```
export const twoFactorSetup = (token: string) =>
  apiFetch<TwoFactorSetupResponse>("/api/auth/2fa/setup", { method: "POST", authToken:
token });

export const twoFactorConfirm = (token: string, code: string) =>
  apiFetch<SigninVerifyResponse>("/api/auth/2fa/confirm",
    { method: "POST", authToken: token, body: JSON.stringify({ code }) });

export const twoFactorLoginVerify = (challengeToken: string, code: string) =>
  apiFetch<SigninVerifyResponse>("/api/auth/2fa/login-verify",
    { method: "POST", body: JSON.stringify({ challengeToken, code }) });
```

РОЗДІЛ 3. ВИПРОБУВАННЯ ТА ОЦІНКА ПІДСИСТЕМИ

3.1 Методика та види тестування

Перевірку коректності підсистеми організовано на кількох рівнях відповідно до видів тестування, передбачених технічним завданням. На рівні функціональних сценаріїв застосовано проходження повних користувацьких шляхів у браузерях на базі рушіїв Chromium та Gecko. На рівні безпеки виконано цільову перевірку типових векторів атак на засоби автентифікації — повторного використання й підроблення токенів, підбору коду та доступу до чутливих даних. На рівні продуктивності та відмовостійкості проведено вимірювання часу відповіді кінцевих точок автентифікації й випробування у штучних умовах недоступності зовнішніх сервісів. Передбачено також інтеграційне, кросбраузерне та адаптивне тестування системи у цілому; у межах підсистеми облікових записів основну увагу приділено функціональному й безпековому випробуванню кінцевих точок.

Методику побудовано за принципом відтворюваності: кожен сценарій формулюється як послідовність кроків із фіксованою очікуваною реакцією системи, що дозволяє повторювати випробування після внесення змін до коду й виявляти регресії. Особливу увагу приділено межовим випадкам — простроченим і повторно застосованим токенам, перевищенню ліміту профілів, спробам видалення кореневого профілю та одночасним зверненням, що моделюють стан гонитви.

3.2 Функціональне випробування

Функціональним випробуванням підтверджено реалізацію всіх вимог технічного завдання. Перевірено такі сценарії: безпарольна реєстрація з автоматичною верифікацією електронної пошти; безпарольний та парольний вхід; відновлення паролю за одноразовим посиланням; багатопрфільність з обмеженням «не більше п'яти профілів» та заборона видалення кореневого профілю; встановлення, зміна й зняття PIN-коду профілю; увімкнення та використання двофакторної автентифікації; платіжно-підписний цикл із резервним режимом; перехід пробного тарифу до обмеження доступу після завершення терміну.

Окремо перевірено коректність переходів скінченного автомата реєстрації зі збереженням уведених даних під час навігації між кроками, автоматичне створення рівно одного кореневого профілю (зокрема за умов подвійного виклику ефектів у режимі розробки) та повторний запит обрання профілю після перезапуску браузера. Підтверджено, що профіль із встановленим PIN-кодом потребує його введення перед обранням, а кореневий профіль здатний скидати PIN-коди дочірніх профілів.

3.3 Безпекове випробування

Безпекове випробування охопило типові вектори атак на засоби автентифікації. Підтверджено, що: повторне використання одноразового посилання відхиляється зі станом «використано»; прострочений токен відхиляється зі станом «прострочено»; підроблення маркера JWT без знання серверного секрету призводить до відмови в авторизації; уведення некоректного PIN-коду чи коду TOTP не надає доступу; запит на відновлення паролю не розкриває наявності облікового запису; дані картки не передаються на сервер системи (підтверджено інспекцією мережесх звернень). Зберігання паролів і одноразових токенів виключно у вигляді криптографічних згорток підтверджено інспекцією таблиць бази даних: у відповідних стовпцях відсутні оригінальні значення. Унаслідок цього компрометація дампу бази даних не надає доступу ні до облікових записів, ні до активних сесій.

3.4 Показники продуктивності та відмовостійкості

Час відповіді кінцевих точок автентифікації за середнього навантаження не перевищує визначеного у технічному завданні порога у 300 мс. Відмовостійкість підтверджено випробуванням у штучних умовах невизначеності: за блокування звернень до платіжного шлюзу клієнт завершує сценарій через резервний режим у межах сумарного тайм-ауту, без «зависання» інтерфейсу; за відсутності налаштованого поштового сервера сценарій реєстрації відтворюється повністю завдяки резервному журнальному режиму, у який записується згенероване посилання. Окремо підтверджено, що скорочений тайм-аут платіжного шлюзу (12 секунд) із одним автоматичним повтором не призводить до тривалого блокування користувацького інтерфейсу.

3.5 Контроль якості та командна взаємодія

Розроблення велося із застосуванням системи контролю версій Git за моделлю гілок, запитів на злиття (pull request) та взаємного рецензування коду, що забезпечило якість інтеграції окремих складових. Типобезпеку клієнтської частини перевіряв компілятор TypeScript, складання — засоби Vite; стиль коду підтримувався статичним аналізатором. Інтеграцію з контентною підсистемою здійснено через узгоджені комунікаційні інтерфейси та спільний контекст автентифікації, що знизило зв'язність складових і спростило паралельну роботу учасників команди.

ВИСНОВКИ

У результаті виконання дипломної роботи спроектовано, реалізовано та випробувано підсистему ідентифікації, керування обліковими записами та платіжно-підписного обслуговування інформаційної системи розповсюдження мультимедійного контенту. Поставлені у вступі задачі виконано повністю; нижче наведено результати в порядку їх формулювання.

1. Проведено порівняльний аналіз провідних систем розповсюдження контенту за сімома ознаками засобів ідентифікації та захисту доступу (табл. 1.1). Встановлено, що жодна із систем-аналогів не поєднує єдиного облікового запису для різно-рідного контенту з двофакторною автентифікацією, PIN-захистом профілю та відмовостійким платіжним потоком; за взірцеву обрано модель багатопрофільного доступу Netflix, а напрямом удосконалення визначено усунення зазначених «мінусів».
2. Синтезовано структурну інформаційну модель облікового запису, профілів і одноразових токенів, нормалізовану до третьої нормальної форми; реалізовано ідемпотентне розгортання схеми засобами `CREATE TABLE IF NOT EXISTS` та умовного додавання стовпців, що усуває потребу ручного супроводу схеми.
3. Реалізовано комунікаційні інтерфейси (контролери автентифікації та профілів) та інфраструктурні сервіси автентифікації, поштового обслуговування й платіжної інтеграції; підсистема є складовою серверної частини з 13 контролерів і 8 інжекттованих сервісів.
4. Впроваджено багаторівневу модель інформаційної безпеки: BCrypt-згортки паролів і PIN-кодів, SHA-256-згортки одноразових токенів, підписаний маркер JWT, безпарольні реєстрацію/вхід/відновлення доступу, PIN-захист профілю та двофакторну автентифікацію за стандартом TOTP, реалізовану власними засобами без зовнішніх залежностей. Підтверджено, що компрометація бази даних не надає доступу ні до акаунтів, ні до активних сесій.
5. Реалізовано клієнтську частину з диференційованим зберіганням стану (довгострокове — для облікового запису, сесійне — для активного профілю) та охоронними компонентами маршрутизації; узгодження клієнта і сервера виконано засобами комунікаційного протоколу REST із підтвердженням маркером JWT. Відтворено Netflix-патерн багатопрофільності з вимогою повторного обрання профілю при кожному новому відвідуванні.
6. Проведено функціональне та безпекове випробування; підтверджено відповідність технічному завданню, час відповіді ендпоінтів автентифікації не перевищує 300 мс, а працездатність зберігається за відмови платіжного та поштового сервісів завдяки градаційному зниженню функціональності.

Новизна роботи полягає не у відтворенні відомого ресурсу, а у створенні вдосконаленого варіанта: поєднано безпарольну реєстрацію, багатопрофільність із PIN-захистом, двофакторну автентифікацію за стандартом TOTP та відмовостійкий платіжний потік у межах єдиного облікового запису. Практична цінність полягає в тому, що обґрунтований шаблон підсистеми облікових записів та результати порівняльного аналізу можуть бути повторно використані наступними розробниками.

Рекомендації та перспективи. Доцільним є впровадження апаратних ключів автентифікації (WebAuthn/FIDO2), інтеграція із зовнішніми провайдерами ідентифікації, а також порівняльне дослідження криптографічних примітивів за критеріями надійності й швидкодії. Загальний висновок проєкту формується як сукупність звітів усіх учасників команди.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. ASP.NET Core documentation [Електронний ресурс] / Microsoft. URL: <https://learn.microsoft.com/aspnet/core/>
2. Entity Framework Core documentation [Електронний ресурс] / Microsoft. URL: <https://learn.microsoft.com/ef/core/>
3. MySQL 8.0 Reference Manual [Електронний ресурс] / Oracle. URL: <https://dev.mysql.com/doc/refman/8.0/en/>
4. RFC 7519. JSON Web Token (JWT) [Електронний ресурс] / M. Jones, J. Bradley, N. Sakimura. IETF, 2015. URL: <https://www.rfc-editor.org/rfc/rfc7519>
5. RFC 6750. The OAuth 2.0 Authorization Framework: Bearer Token Usage [Електронний ресурс] / M. Jones, D. Hardt. IETF, 2012. URL: <https://www.rfc-editor.org/rfc/rfc6750>
6. RFC 6238. TOTP: Time-Based One-Time Password Algorithm [Електронний ресурс] / D. M'Raihi et al. IETF, 2011. URL: <https://www.rfc-editor.org/rfc/rfc6238>
7. RFC 4648. The Base16, Base32, and Base64 Data Encodings [Електронний ресурс] / S. Josefsson. IETF, 2006. URL: <https://www.rfc-editor.org/rfc/rfc4648>
8. RFC 7231. HTTP/1.1: Semantics and Content [Електронний ресурс] / R. Fielding, J. Reschke. IETF, 2014. URL: <https://www.rfc-editor.org/rfc/rfc7231>
9. OWASP Authentication Cheat Sheet [Електронний ресурс] / OWASP Foundation. URL: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
10. RFC 2898. PKCS #5: Password-Based Cryptography Specification [Електронний ресурс] / B. Kaliski. IETF, 2000. URL: <https://www.rfc-editor.org/rfc/rfc2898>
11. PCI DSS v4.0 [Електронний ресурс] / PCI Security Standards Council, 2022. URL: <https://www.pcisecuritystandards.org/>
12. OWASP Application Security Verification Standard (ASVS) v4.0.3 [Електронний ресурс] / OWASP Foundation, 2021. URL: <https://owasp.org/www-project-application-security-verification-standard/>
13. Stripe API Reference [Електронний ресурс] / Stripe. URL: <https://stripe.com/docs/api>
14. Stripe.NET library [Електронний ресурс] / Stripe. URL: <https://github.com/stripe/stripe-dotnet>
15. MailKit documentation [Електронний ресурс] / J. Stedfast. URL: <https://github.com/jstedfast/MailKit>
16. React documentation [Електронний ресурс] / Meta Open Source. URL: <https://react.dev/learn>
17. TypeScript documentation [Електронний ресурс] / Microsoft. URL: <https://www.typescriptlang.org/docs/>

18. Vite documentation [Электронный ресурс]. URL: <https://vite.dev/guide/>
 19. React Router documentation [Электронный ресурс]. URL: <https://reactrouter.com/>
 20. TanStack Query documentation [Электронный ресурс]. URL: [https://tanstack.com/
query/latest](https://tanstack.com/query/latest)
-

ДОДАТОК А. ТЕХНІЧНЕ ЗАВДАННЯ ПРОЄКТУ

Загальний опис

Програмне забезпечення реалізує розподілену інформаційну систему розповсюдження мультимедійного контенту за моделлю підписки (робоча назва — Bazinga). Система надає кінцевим користувачам доступ до двох категорій контенту — коміксів/манги (Bazinga Comics) та потокового відео (BazingaTV) — у межах єдиного облікового запису. Цей звіт присвячено підсистемі ідентифікації, керування обліковими записами та платіжно-підписного обслуговування, яка є спільною основою обох категорій контенту.

Призначення

Програмне забезпечення призначене для надання користувачеві можливості зареєструватися, увійти, керувати багатoproфільним обліковим записом, оформити підписку та платіжний інструмент, а також безпечно завершити сесію. Підсистема забезпечує захист персональних і платіжних даних та коректну роботу за відмови окремих зовнішніх сервісів.

Цілі та задачі системи

Система повинна забезпечувати: реєстрацію, автентифікацію та авторизацію користувачів із запобіганням повторній реєстрації; безпарольний вхід та відновлення доступу; двофакторну автентифікацію; багатoproфільність з PIN-захистом; оформлення та супровід підписки; керування персональними даними; захист персональних і платіжних даних.

Функціональні вимоги

1. Реєстрація, автентифікація та авторизація. Користувач повинен мати можливість: зареєструватися за електронною поштою за безпарольним сценарієм; підтвердити реєстрацію переходом за одноразовим посиланням; увійти за паролем або за одноразовим посиланням; увімкнути двофакторну автентифікацію; відновити пароль за одноразовим посиланням; вийти з акаунта. Обмеження та перевірки: валідація електронної пошти; перевірка мінімальної складності паролю; обмеження терміну дії одноразових посилань (15 хвилин) та їх одноразовість; обмеження терміну дії маркера сесії.

2. Профіль користувача. Користувач повинен мати можливість: створювати до п'яти профілів; редагувати ім'я та аватар профілю; позначати профіль як дитячий; встановлювати, змінювати та знімати PIN-код профілю; обирати активний профіль при кожному новому відвідуванні. Кореневий профіль не може бути видалений; кореневий профіль може скидати PIN-коди дочірніх профілів.

3. Підписка та оплата. Система повинна забезпечувати: обрання тарифного плану (Bazinga Comics, BazingaTV, Bazinga Unlimited або пробний); прив'язку платіжного інструменту виключно через офіційний компонент платіжного шлюзу; автоматичний перехід пробного тарифу до обмеження доступу після завершення; резервний режим оформлення за недоступності платіжного шлюзу.

4. Особистий кабінет. Користувач повинен мати можливість: переглядати й редагувати персональні дані та номер телефону; переглядати відомості про підписку; змінювати пароль; керувати двофакторною автентифікацією та PIN-кодами профілів.

Нефункціональні вимоги

1. Продуктивність. Час відповіді ендпоінтів автентифікації — не більше 300 мс за стандартного навантаження; кешування часто запитуваних даних.

2. Масштабованість. Stateless-архітектура серверної частини з можливістю горизонтального масштабування; дотримання правил REST.

3. Безпека. Зберігання паролів і PIN-кодів виключно у вигляді BCrypt-згорток; зберігання одноразових токенів виключно у вигляді SHA-256-згорток; підпис маркера сесії алгоритмом HMAC SHA-256; передавання маркера за схемою Bearer; обмеження кількості та терміну дії одноразових посилань; мінімізація області застосування стандарту PCI DSS.

4. Адаптивність. Коректна робота у браузерях Google Chrome, Mozilla Firefox, Safari, Microsoft Edge та у мобільних браузерах; адаптація інтерфейсу під десктопи, планшети та смартфони.

Тестування

Передбачено функціональне, інтеграційне, безпекове, кросбраузерне та адаптивне тестування; у межах підсистеми облікових записів основну увагу приділено функціональному та безпековому випробуванню кінцевих точок.

Критерії готовності

Проект вважається виконаним, якщо: реалізовано всі функціональні вимоги; відсутні критичні помилки; підсистема коректно працює на всіх погоджених пристроях і браузерах; виконано вимоги щодо безпеки; успішно проведено тестування; складено звітну частину.